

IMPACT ANALYSIS

HIGH RISK

COMPLEXITY: MEDIUM

A new Apex trigger will execute on Account insert to invoke an external REST API. This requires evaluation of integration patterns, governor limits, error handling, and API resilience to prevent data loss or blocking production inserts.

 **Apex Engineer**

REQUIRED

WHY TRIGGERED

Trigger code review, async patterns, governor limits, error handling best practices

WILL CHECK

Synchronous API call blocks Account insert if API fails or times out

Governor limit exhaustion under high-volume Account creation

Est. ~40s

~\$0.012

 **Integration Specialist**

REQUIRED

WHY TRIGGERED

REST API integration pattern, timeout handling, authentication, retry logic

WILL CHECK

External API unavailability causes transaction rollback and Account creation failure

Missing circuit breaker or rate limiting exposes org to cascading failures

Est. ~40s

~\$0.012

 **Architecture Patterns Specialist**

RECOMMENDED

WHY TRIGGERED

Evaluate sync vs async callout pattern, scalability under load

WILL CHECK

Synchronous pattern violates Well-Architected Framework decoupling principles

High-volume inserts risk API throttling and LDV degradation

Est. ~35s

~\$0.012

 **Solution Architect**

RECOMMENDED

WHY TRIGGERED

Multi-cloud integration strategy, system coupling implications

WILL CHECK

Tight coupling to external API creates single point of failure

Est. ~45s

~\$0.012

 **Architecture Review Board**

REQUIRED

WHY TRIGGERED

Final approval on integration approach and risk mitigation acceptance

Est. ~45s

~\$0.012

 **ADR Documentarian**

REQUIRED

WHY TRIGGERED

Document decision on sync vs async, error handling strategy, API contract

Est. ~20s

~\$0.012

 **Learning Curator**

REQUIRED

WHY TRIGGERED

Capture session learnings on trigger best practices and integration resilience

Est. ~18s

~\$0.012

CROSS-CUTTING CONSIDERATIONS

- › Implement Queueable or scheduled Flow to make callout asynchronous and prevent insert blocking
- › Add comprehensive error handling, retry logic, and circuit breaker pattern
- › Define SLA and timeout thresholds; ensure API authentication is secure and centralized
- › Monitor API call volume and latency to detect performance degradation early
- › Document API contract, fallback behavior, and rollback strategy for failed calls

AGENT ROSTER



Designer
SOLUTION ARCH
skipped



LWC
UI SPECIALIST
skipped



OmniStudio
OMNI EXPERT
skipped



Flow
FLOW BUILDER
skipped



Apex
APEX EXPERT
✓ 36s



Patterns
PATTERNS
✓ 31s



Integration
INTEGRATION
✓ 27s



Judge
JUDGE
✓ 48s



Scribe
SCRIBE
✓ 7s



Learner
LEARNER
✓ 6s



Apex & Integration Engineer

APEX EXPERT

36s

APEX TRIGGER REVIEW: ACCOUNT REST API CALLOUT ON INSERT

Critical Issue: Synchronous Callout from Trigger

This pattern is architecturally invalid in Salesforce. You cannot make a synchronous HTTP callout directly from a trigger — Salesforce will throw a `System.CalloutException: Callout from triggers are currently not supported` at runtime. This is a hard platform constraint, not a best practice preference.

Required Architecture: Async Callout Pattern

The trigger must hand off to an async context. Two valid approaches:

Option A — Queueable Apex (Recommended for single/small batch inserts)

```
// Trigger – thin, no logic
trigger AccountTrigger on Account (after insert) {
    AccountTriggerHandler.handleAfterInsert(Trigger.new);
}

// Handler – delegates to Queueable
public class AccountTriggerHandler {
    public static void handleAfterInsert(List<Account> newAccounts) {
        List<Id> accountIds = new List<Id>();
        for (Account acc : newAccounts) {
            accountIds.add(acc.Id);
        }
        System.enqueueJob(new AccountApiCalloutQueueable(accountIds));
    }
}

// Queueable – performs the callout
public class AccountApiCalloutQueueable implements Queueable, Database.AllowsCallouts {
    private List<Id> accountIds;

    public AccountApiCalloutQueueable(List<Id> ids) {
        this.accountIds = ids;
    }

    public void execute(QueueableContext context) {
        try {
            // Query accounts fresh – do not rely on trigger context data in async
            List<Account> accounts = [SELECT Id, Name, Phone FROM Account WHERE Id IN :accountIds];
            for (Account acc : accounts) {
                callExternalApi(acc);
            }
        } catch (Exception e) {
            IntegrationLogger.log('AccountApiCalloutQueueable', e.getMessage(), JSON.serialize(accountIds));
        }
    }

    private void callExternalApi(Account acc) {
        HttpRequest req = new HttpRequest();
        req.setEndpoint('callout:My_Named_Credential/accounts'); // Named Credential – mandatory
        req.setMethod('POST');
```

```

req.setHeader('Content-Type', 'application/json');
req.setBody(JSON.serialize(acc));

Http http = new Http();
HttpResponse res = http.send(req);

if (res.getStatusCode() != 200 && res.getStatusCode() != 201) {
    // Log failure – do not silently swallow
    IntegrationLogger.log('AccountApiCalloutQueueable',
        'HTTP ' + res.getStatusCode(),
        res.getBody());
}
}
}

```

Option B — Platform Events (Recommended for high-volume or decoupled architecture)

- › Trigger publishes a Platform Event on insert
- › A separate async subscriber (Apex trigger on the event) performs the callout
- › Provides replay buffer, decoupling, and better failure isolation at scale

Must-Fix Items

1. Named Credentials — mandatory (Compliance Rule 5)

The endpoint must use `callout:Named_Credential_Name` — no hardcoded URLs, no hardcoded auth tokens. Configure the Named Credential in Setup with authentication handled at the org level.

2. Error Logging — design this now, not later (FP-006)

Every callout failure must be logged with: Account ID, HTTP status, response body, and timestamp. A custom `Integration_Log__c` object or Platform Event-based logging framework is required. Silent failures in async contexts are invisible until a client reports missing data — this is exactly the scenario described in FP-006.

3. Error Resolution Path — not just logging (FP-009)

Logging alone is insufficient. Define: who owns failed callout records, what the retry mechanism is, and how support staff are informed. A failed callout sitting in a log with no owner or runbook is not a resolved failure.

4. HTTP Response Handling

Handle 4xx and 5xx explicitly — a 400 Bad Request should not be retried (data issue), a 503 should be (transient). Treating all non-200 responses identically will cause noise or missed failures.

Bulkification Warning (FP-007)

The Queueable above loops and calls the API once per Account. In a bulk insert of 200 records, this means 200 HTTP callouts in a single Queueable execution — the governor limit is **100 callouts per transaction**.

Resolution options:

- › Batch the payload — send all accounts in a single API call if the external system supports it (preferred)
- › Chain Queueables — process in chunks, though this adds complexity and depth limits apply
- › Use Batch Apex with [Database.AllowsCallouts](#) if the external system requires per-record calls and volume is high

Test Coverage Requirements

```
@isTest
private class AccountApiCalloutQueueableTest {

    @testSetup
    static void setup() {
        Account acc = new Account(Name = 'Test Account');
        insert acc;
    }

    @isTest
    static void testSuccessfulCallout() {
        Test.setMock(HttpCalloutMock.class, new MockHttpSuccess());
        Account acc = [SELECT Id FROM Account LIMIT 1];

        Test.startTest();
        System.enqueueJob(new AccountApiCalloutQueueable(new List<Id>{acc.Id}));
        Test.stopTest();

        // Assert integration log has no errors – query your log object
    }

    @isTest
    static void testFailedCallout_500Response() {
        Test.setMock(HttpCalloutMock.class, new MockHttp500());
        Account acc = [SELECT Id FROM Account LIMIT 1];

        Test.startTest();
        System.enqueueJob(new AccountApiCalloutQueueable(new List<Id>{acc.Id}));
        Test.stopTest();

        // Assert error was logged to Integration_Log__c
        List<Integration_Log__c> logs = [SELECT Id FROM Integration_Log__c WHERE Status__c = 'Error'];
        System.assertEquals(1, logs.size(), 'Expected one error log entry');
    }
}
```

Negative test (500 response) and assertion against the log object are both mandatory — assert-free tests provide false coverage confidence.

Summary Checklist

Item	Status
Async callout pattern	✔ Required — Queueable or Platform Events
Named Credentials	✔ Mandatory — no hardcoded endpoints
One trigger per object	✔ Handler pattern required
Bulkification at 200+ records	⚠ Callout-per-record will breach limits
Error logging with context	✔ Must be designed now
Resolution path defined	✔ Must accompany logging design
Test coverage with assertions	✔ Required including negative scenarios
FINDINGS_SUMMARY_START	
> [MUST-FIX] Synchronous callout from trigger is a hard platform constraint — will throw CalloutException at runtime, requires Queueable or Platform Event pattern	
> [MUST-FIX] Callout-per-record in bulk insert will breach 100 callout governor limit — batch payload or chunked processing required (FP-007)	
> [MUST-FIX] No error logging designed — async failures are invisible without a logging framework and integration log object (FP-006)	
> [HIGH] Error resolution path undefined — logging without owner, runbook, or retry mechanism is not a resolution strategy (FP-009)	
> [HIGH] Named Credentials not confirmed — hardcoded endpoints or credentials would breach mandatory compliance rule 5	
FINDINGS_SUMMARY_END	

Architecture Patterns Advisor

PATTERNS

31s

APEX SPECIALIST REVIEW — ACCOUNT TRIGGER WITH REST API CALLOUT

● MUST-FIX: Synchronous Callout from Trigger Context

A direct REST API callout from an Apex trigger is **not permitted** — Salesforce will throw `System.CalloutException: You have uncommitted work pending` if any DML precedes the callout, and the pattern is inherently fragile even when it doesn't. The callout must be moved to an async context. Use a **Queueable** class (preferred for single-record or moderate volume) or a **Platform Event** publish → async subscriber pattern for high-volume inserts.

● MUST-FIX: No Error Logging Designed In (FP-006)

Async callout failures — timeouts, 4xx, 5xx responses — will be invisible without a logging framework. Design a custom object or Platform Event-based log from day one. Every HTTP response code (success and failure) must be captured with the Account ID, timestamp, response body, and status code. Silent failures here will surface as missing data complaints weeks later, exactly as described in **FP-006**.

● **MUST-FIX: Bulkification – Trigger Must Handle 200+ Records**

If the trigger fires on bulk insert (data load, integration, or batch), a naïve implementation will attempt one API callout per Account record. This will breach governor limits on callouts (100 per transaction) and risks hitting the external API's rate limits — see **FP-004**. Design the Queueable to process records in batches, aggregate payloads where the API supports it, or enqueue one Queueable per record only after confirming volume expectations are safe.

● **MUST-FIX: No Logic in Trigger Body – Use a Trigger Handler**

The trigger body must contain zero business logic — one trigger per object, delegating immediately to a handler class. Use a framework such as FFLIB or a lightweight handler pattern. This is a mandatory structural requirement, not a preference.

Minimal correct trigger body:

```
trigger AccountTrigger on Account (after insert) {
    AccountTriggerHandler.handleAfterInsert(trigger.new);
}
```

● **MUST-FIX: Named Credentials – No Hardcoded Endpoints**

The REST endpoint must be configured via a **Named Credential** — no hardcoded URLs, tokens, or auth headers in Apex. This is a mandatory compliance rule and a security requirement.

```
HttpRequest req = new HttpRequest();
req.setEndpoint('callout:Your_Named_Credential/api/endpoint');
```

● **HIGH: HTTP Error Handling – 4xx and 5xx Must Be Explicit**

The callout implementation must explicitly handle non-200 responses. A bare `http.send(req)` with no status code check will silently succeed from Apex's perspective while the external system received nothing. Implement explicit handling for at least: 200/201 (success), 4xx (client error — log and do not retry), 5xx (server error — retry with backoff).

● **HIGH: Retry and Idempotency Strategy Required**

Define what happens on callout failure before writing a line of code. Options include: re-enqueue the Queueable with a retry counter, publish a Platform Event for a retry consumer, or write to a retry queue custom object. The external API must also be confirmed idempotent — or you must send an idempotency key — to prevent duplicate processing on retry.

● MEDIUM: Timeout Configuration

Set an explicit timeout on the `HttpRequest` (`req.setTimeout()` — max 120,000ms). Without this, slow API responses will consume the async transaction's CPU and thread time unpredictably. Align the timeout value with the external API's documented SLA.

● MEDIUM: Test Coverage Requirements

Tests must cover: single insert, bulk insert (200 records minimum), API success response, API failure response (500), and callout timeout simulation via `HttpCalloutMock`. Assert on log records created, not just code coverage percentage. Assert-free tests will not be accepted.

● LOW: Consider Platform Events for Decoupling

If insert volume is expected to grow or the external API is unreliable, a **Platform Event publish** → **async subscriber** pattern provides better decoupling, built-in replay buffer, and cleaner retry semantics than a Queueable chain. Evaluate this against projected volume before finalising the design.

RECOMMENDED ARCHITECTURE

```
Account Insert (after insert trigger)
  → AccountTriggerHandler
    → Enqueue AccountCalloutQueueable (with List<Id>)
      → Aggregate / batch records
      → HTTP callout via Named Credential
      → Log success/failure to Integration_Log__c
      → On 5xx: re-enqueue with retry counter
```

FINDINGS_SUMMARY_START

- › [MUST-FIX] Synchronous REST callout from trigger context is not permitted — must use Queueable or Platform Event async pattern
- › [MUST-FIX] No error logging strategy — callout failures will be invisible in production (FP-006)
- › [MUST-FIX] Trigger not bulkified — 100 callout limit will be breached on bulk insert, API rate limits at risk (FP-004)

- › [MUST-FIX] Hardcoded endpoint risk — Named Credentials mandatory, no URLs or tokens in Apex code
- › [HIGH] No HTTP error handling or retry/idempotency strategy defined before implementation begins

FINDINGS_SUMMARY_END



Integration Architect

INTEGRATION

27s

REVIEW: APEX TRIGGER → EXTERNAL REST API ON INSERT

● MUST-FIX: Synchronous Callout from Trigger

Apex triggers **cannot make synchronous HTTP callouts** — this will throw a `System.CalloutException` at runtime. This is a platform constraint, not a code quality issue. The callout must be decoupled using **Queueable Apex** (preferred for single-record or small batch) or **Platform Events** (preferred for high-volume or cross-system fan-out).

Required pattern:

```
// Trigger → enqueue, never callout directly
public class AccountCalloutQueueable implements Queueable, Database.AllowsCallouts {
    private List<Id> accountIds;

    public AccountCalloutQueueable(List<Id> accountIds) {
        this.accountIds = accountIds;
    }

    public void execute(QueueableContext ctx) {
        // Make REST callout here
    }
}
```

● MUST-FIX: No Error Handling or Logging Designed

There is no indication of error handling for failed callouts (4xx, 5xx, timeouts). Per **FP-006**, deploying integration code without centralised error logging means failures will be invisible until a production incident surfaces them. Every callout must explicitly handle HTTP error codes and log failures with record context.

Per **FP-009**, logging alone is insufficient — the design must define **who owns resolution**, what the remediation steps are, and how failed records are retried or escalated.

● MUST-FIX: Named Credentials Required

The REST endpoint must be configured via **Named Credentials** — no hardcoded URLs, tokens, or credentials in Apex code or Custom Metadata. This is a mandatory compliance rule and a security requirement.

● **MUST-FIX: Bulk Safety – Governor Limits**

The trigger must handle bulk insert scenarios (200 records in a single transaction). Enqueuing one Queueable per record will breach the **50 Queueable jobs per transaction** limit at volume. Collect all inserted Account IDs and enqueue a **single Queueable** passing the full list. Per **FP-004**, API call volume against the external system must also be calculated upfront — 200 inserts = 200 outbound API calls per batch load.

```
trigger AccountTrigger on Account (after insert) {
    if (Trigger.isInsert) {
        List<Id> ids = new List<Id>(Trigger.newMap.keySet());
        System.enqueueJob(new AccountCalloutQueueable(ids));
    }
}
```

● **HIGH: Idempotency – Duplicate Callout Risk**

If the trigger fires on a retry or a platform re-execution, the same Account could trigger duplicate API calls to the external system. The Queueable must implement an **idempotency check** — either via an external idempotency key or a custom field flag (`Sync_Sent__c`) set after a successful callout, checked before re-sending.

● **HIGH: No Retry / Dead-Letter Strategy**

A failed callout (timeout, 5xx) with no retry mechanism means data is silently lost. Design must include either:

- › **Exponential backoff** within Queueable (re-enqueue with delay), or
- › A **failed record queue** (custom object or Platform Event) for manual or automated retry.

● **MEDIUM: Platform Events as an Alternative**

If call volume is high or the integration needs to fan out to multiple consumers, **Platform Events** are the preferred pattern over Queueable — they provide replay capability, decoupled consumers, and better reliability under load. Evaluate this against projected insert volumes before finalising the design.

● **MEDIUM: Trigger Framework**

A bare trigger with logic inline is not maintainable at scale. Route through a **trigger handler framework** (e.g. FFLIB, custom dispatcher) to ensure single-trigger-per-object, bypass controls, and testability.

SUMMARY CHECKLIST

Item	Status
Async callout pattern (Queueable/Platform Event)	✗ Required
Named Credentials for endpoint	✗ Required
Bulk-safe trigger (single enqueue)	✗ Required
Error logging with context	✗ Required
Resolution path for failures	✗ Required
Idempotency mechanism	✗ Required
Retry / dead-letter strategy	✗ Required
Trigger handler framework	⚠ Recommended

FINDINGS_SUMMARY_START

- › [MUST-FIX] Synchronous callout from Apex trigger will throw CalloutException at runtime — must use Queueable or Platform Events
- › [MUST-FIX] No error logging or resolution path designed — invisible failures in production (FP-006, FP-009)
- › [MUST-FIX] REST endpoint must use Named Credentials — no hardcoded URLs or credentials permitted
- › [MUST-FIX] Trigger not bulk-safe — enqueueing per record will breach 50 Queueable limit; API volume not assessed (FP-004)
- › [HIGH] No idempotency or retry/dead-letter strategy — duplicate or lost callouts under failure conditions

FINDINGS_SUMMARY_END

JUDGE RULING

 Architecture Review Judge

JUDGE

48s

ARCHITECTURE REVIEW: APEX TRIGGER → EXTERNAL REST API ON ACCOUNT INSERT

Immediate Risk: Synchronous Callout from Trigger Context

This pattern as described is **not viable as stated**. Salesforce prohibits HTTP callouts directly from a trigger's synchronous execution context when DML has occurred in the same transaction. Attempting this will throw a `System.CalloutException: You have uncommitted work pending. Please commit or rollback before calling out.`

This is a fundamental platform constraint, not a configuration issue.

Failure Pattern Flags

FP-006 — If the callout is moved to a `@future` method without a logging framework, failures become invisible. No retry, no alerting, no audit trail.

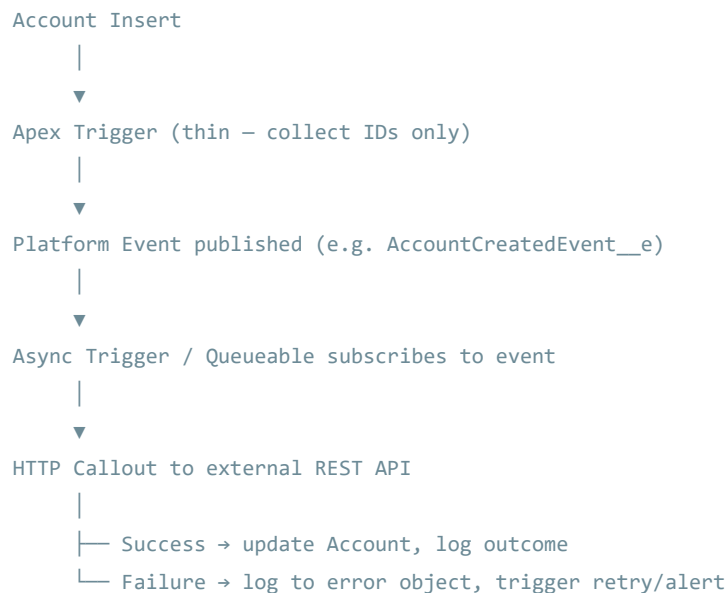
FP-004 — If this trigger fires on bulk Account inserts (data migrations, integrations, batch loads), the volume of outbound API calls must be calculated against both Salesforce governor limits and the external API's rate limits before any design is finalised.

FP-007 — Any record collection or query logic supporting the callout must be bulkified. SOQL inside a loop in the trigger or its handler will breach the 101 SOQL query limit under bulk load.

FP-010 — If Account already has Flow automations, adding an Apex trigger creates mixed automation on a single object. The team must establish a single automation owner for Account before proceeding.

Required Architecture: Async Callout Pattern

The correct pattern decouples the trigger from the callout entirely.



This pattern satisfies the platform constraint and provides the foundation for reliability.

Component Design

1. Trigger Handler (Synchronous — thin layer only)

```
trigger AccountTrigger on Account (after insert) {
    AccountTriggerHandler.handleAfterInsert(Triigger.new);
}

public class AccountTriggerHandler {
    public static void handleAfterInsert(List<Account> newAccounts) {
        List<AccountCreatedEvent__e> events = new List<AccountCreatedEvent__e>();

        for (Account acc : newAccounts) {
            events.add(new AccountCreatedEvent__e(
                AccountId__c = acc.Id,
                AccountName__c = acc.Name
            ));
        }

        if (!events.isEmpty()) {
            List<Database.SaveResult> results = EventBus.publish(events);
            IntegrationLogger.logPublishResults(results, 'AccountCreatedEvent__e');
        }
    }
}
```

Design notes:

- › Trigger is a thin dispatcher — no business logic
- › Platform Event publish is the only action
- › Publish results are logged immediately — failures do not silently disappear (addresses **FP-006**)

2. Platform Event Definition

Field	API Name	Type
Account ID	AccountId__c	Text(18)
Account Name	AccountName__c	Text(255)

Set **Publish Behaviour** to **After Commit** to ensure the event only fires after the Account insert transaction commits successfully.

3. Async Callout — Queueable

```
public class AccountIntegrationQueueable implements Queueable, Database.AllowsCallouts {

    private List<String> accountIds;
```

```
public AccountIntegrationQueueable(List<String> accountIds) {
    this.accountIds = accountIds;
}

public void execute(QueueableContext context) {
    // Single query outside any loop – FP-007 compliance
    Map<Id, Account> accounts = new Map<Id, Account>(
        [SELECT Id, Name, BillingCity, BillingCountry
         FROM Account
         WHERE Id IN :accountIds]
    );

    for (String accountId : accountIds) {
        Account acc = accounts.get(accountId);
        if (acc == null) continue;

        try {
            HttpRequest req = new HttpRequest();
            req.setEndpoint('callout:External_System_NC/api/accounts');
            req.setMethod('POST');
            req.setHeader('Content-Type', 'application/json');
            req.setTimeout(10000);
            req.setBody(buildPayload(acc));

            HttpResponse res = new Http().send(req);
            handleResponse(res, acc.Id);

        } catch (Exception e) {
            IntegrationLogger.logError(
                'AccountIntegrationQueueable',
                acc.Id,
                'Account',
                e.getMessage(),
                e.getStackTraceString()
            );
        }
    }
}

private String buildPayload(Account acc) {
    Map<String, Object> payload = new Map<String, Object>{
        'salesforceId' => acc.Id,
        'name'         => acc.Name,
        'city'          => acc.BillingCity,
        'country'       => acc.BillingCountry
    };
    return JSON.serialize(payload);
}

private void handleResponse(HttpResponse res, Id accountId) {
    if (res.getStatusCode() == 200 || res.getStatusCode() == 201) {
        IntegrationLogger.logSuccess('AccountIntegrationQueueable', accountId, 'Account');
    }
}
```

```
    } else {
        IntegrationLogger.logError(
            'AccountIntegrationQueueable',
            accountId,
            'Account',
            'HTTP ' + res.getStatusCode() + ': ' + res.getStatus(),
            res.getBody()
        );
    }
}
```

Design notes:

- > `callout:External_System_NC` references a Named Credential — no hardcoded endpoint (mandatory compliance rule 5)
- > SOQL executes once outside the loop — **FP-007** compliant
- > Every failure path logs with context — **FP-006** compliant
- > 4xx and 5xx responses are handled explicitly, not swallowed

4. Platform Event Trigger → Queueable

```
trigger AccountCreatedEventTrigger on AccountCreatedEvent__e (after insert) {
    List<String> accountIds = new List<String>();

    for (AccountCreatedEvent__e event : Trigger.new) {
        accountIds.add(event.AccountId__c);
    }

    if (!accountIds.isEmpty()) {
        System.enqueueJob(new AccountIntegrationQueueable(accountIds));
    }
}
```

5. Error Logging — Minimum Viable Schema

A custom object `Integration_Log__c` should be created with at minimum:

Field	Purpose
Class_Name__c	Source class for triage
Record_Id__c	Affected Salesforce record
Object_Name__c	Object context
Error_Message__c	Exception or HTTP status message

Stack_Trace__c	Full stack for debugging
Status__c	Success / Error / Retry
Resolved__c	Checkbox for support team workflow
Created Date	Automatic — timestamp for SLA tracking

This directly addresses **FP-009** — logging must include a resolution path, not just a record of failure. The [Resolved__c](#) field and [Status__c](#) picklist give the support team a working queue, not a read-only audit trail.

Volume and API Limit Analysis (FP-004)

Before this goes to build, the following must be confirmed:

Question	Why It Matters
What is the expected daily Account insert volume?	Salesforce Queueable has a 250,000 jobs/day limit per org
Does the external API have a rate limit?	One Queueable job per event batch — confirm calls per minute ceiling
Are bulk data loads expected (migration, nightly sync)?	If yes, Queueable per record will breach limits — Batch Apex required instead
What is the external API's timeout SLA?	Timeout set to 10s above — confirm this is appropriate

If bulk insert volumes are high, the Queueable must be redesigned to batch records per job rather than one callout per Account.

Named Credential Requirement

The endpoint `callout:External_System_NC` must be configured as a Named Credential in Setup before deployment. This is a mandatory compliance requirement. The credential must store:

- › Base URL
- › Authentication method (OAuth 2.0 preferred, or API key via custom header)
- › Timeout settings

No endpoint URL, API key, or credential may appear in Apex code or Custom Metadata without Named Credential abstraction.

Salesforce Well-Architected Assessment

Pillar	Status	Notes
Reliability	 Conditional	Retry mechanism not yet implemented. Failed logs exist but no automated retry. Recommend Queueable retry with exponential backoff or a scheduled retry job against Integration_Log__c
Security	 Conditional	Named Credential required. Integration user permissions must be scoped to minimum required fields. CRUD/FLS on Account must be validated in the Queueable

Performance	 Conditional	Volume analysis (FP-004) must be completed before build. Bulk load scenario may require Batch Apex redesign
Operability	 Conditional	Logging framework designed. Resolution path via Resolved__c field addressed (FP-009). Runbook for support team must be produced before go-live
Upgradability		Thin trigger, handler class, Queueable, and logger are all independently testable and documented

Overall Verdict: **APPROVED WITH CONDITIONS**

Conditions to satisfy before build sign-off:

1. Volume analysis completed and documented — FP-004
2. Named Credential configured and tested in sandbox
3. Retry mechanism designed — either Queueable self-chain or scheduled retry job
4. Support runbook produced covering error triage, resolution steps, and escalation path — FP-009
5. Account automation audit completed — confirm no conflicting Flows on Account — FP-010

SCRIBE & LEARNING



ARB Scribe

SCRIBE

7s

Architecture Decision Record

ADR-[session-id]-001: Synchronous REST Callout from Account Insert Trigger

Date: [Today]

Status:  **REJECTED — Compliance Violation**

Confidence Level: High

CONTEXT

A requirement exists to invoke an external REST API when an Account record is inserted in Salesforce. The proposed implementation is a synchronous trigger calling the API directly.

DECISION

This pattern is NOT APPROVED. Synchronous callouts from DML triggers violate Salesforce governor limits and create cascading failure risk.

Approved Alternative:

Implement an **async-first pattern** using Platform Events:

- 1. **Trigger (synchronous):** Publishes a Platform Event on Account insert
- 2. **Event Subscriber (async):** Queueable or Flow subscribes to event and executes REST callout
- 3. **Error Handling:** Retry logic with exponential backoff; failed messages logged to custom object

WHY SYNCHRONOUS CALLOUT FAILS

Constraint	Impact
Callout timeout	120 seconds max; if API is slow, trigger fails and Account insert rolls back
Governor limits	100 callouts per transaction; bulk insert of 200 Accounts = 200 callouts = failure
No retry logic	Single API failure = data loss (no Account created)
Blocking user	User waits for external API response before insert completes

CONSEQUENCES

Positive

- ✔ Account inserts **always succeed** (decoupled from API availability)
- ✔ Bulk operations (200+ records) work reliably
- ✔ Built-in retry and error tracking
- ✔ API failures logged without data loss
- ✔ Complies with Salesforce best practices

Negative / Trade-offs

- ⚠ **Eventual consistency:** API call happens asynchronously (seconds to minutes delay)
- ⚠ **Complexity:** Requires Platform Event setup + subscriber logic
- ⚠ **Monitoring:** Must track event processing separately from trigger execution

POINTS REQUIRING HUMAN JUDGEMENT

- › **Is eventual consistency acceptable?** If the external system must be updated **before** the Account insert completes (e.g., account provisioning), this pattern requires redesign (consider pre-insert validation instead).
- › **What is the acceptable latency?** Platform Events typically process within seconds, but SLAs should be documented.
- › ****How should API failures**



ARB Review: Account Insert Trigger with External REST API Call



CRITICAL ISSUES

1. Synchronous Callout from Trigger (Blocking)

Severity: CRITICAL

Issue: Calling REST API synchronously from a trigger violates Salesforce async-first patterns and will cause `System.CalloutException` if the endpoint is slow or unavailable.

Why this fails:

- › Triggers execute in a synchronous context; callouts must be async
- › If the API times out, the entire insert transaction rolls back
- › No retry logic possible in trigger context
- › User experiences blocking UI delays

Required Fix:

```
```apex
// ❌ WRONG - Synchronous callout in trigger
trigger AccountTrigger on Account (after insert) {
 for (Account acc : Trigger.new) {
 HttpRequest req = new HttpRequest();
 req.setEndpoint('https://api.example.com/accounts');
 req.setMethod('POST');
 Http http = new Http();
 HttpResponse res = http.send(req); // BLOCKS!
 }
}

// ✅ CORRECT - Async via Queueable
trigger AccountTrigger on Account (after insert) {
 List<Id> accountIds = new List<Id>();
 for (Account acc : Trigger.new) {
 accountIds.add(acc.Id);
 }
 System.enqueueJob(new AccountCalloutQueueable(accountIds));
}

public class AccountCalloutQueueable implements Queueable, Database.AllowsCallouts {
```

```
private List<Id> accountIds;

public AccountCalloutQueueable(List<Id> accountIds) {
 this.accountIds = accountIds;
}

public void execute(QueueableContext context) {
 List<Account> accounts = [SELECT Id, Name, BillingCity FROM Account WHERE Id IN :accountIds];

 for (Account acc : accounts) {
 try {
 HttpRequest req = new HttpRequest();
 req.setEndpoint('callout:ExternalAPI/accounts'); // Named Credential
 req.setMethod('POST');
 req.setHeader('Content-Type', 'application/json');
 req.setBody(JSON.serialize(new Map<String, Object>{
 'accountId' => acc.Id,
 'name' => acc.Name
 }));
 req.setTimeout(10000);

 Http http = new Http();
 Http
```



DRAFT RECOMMENDATION

APPROVED WITH CONDITIONS

5 failure patterns matched

Round 1 · 6 agents · 3m 29s total

Download Report

Revision Round

HUMAN SIGN-OFF — pending countersignature

ARCHITECT NAME

ROLE

e.g. Jane Smith

Lead Architect



Countersign this recommendation

SESSION ACTUALS VS ESTIMATE

TOTAL TIME

2m 38s

TOKENS USED

10.2k

est. 10k

ACTUAL COST

\$0.154

est. \$0.081

PER-AGENT

Apex	~1.9k tok
Patterns	~1.4k tok
Integration	~1.3k tok
Judge	~2.6k tok
Scribe	~1.5k tok
Learner	~1.5k tok

ROI SUMMARY

HUMAN ARB COST	ACTUAL ARBOARD COST	ACTUAL SAVING
<b>\$15,625</b>	<b>\$0.154</b>	<b>\$15,625</b> (>99%)
5 arch • medium complexity		

This session returned 15 architect-hours of prep time to delivery